

컴퓨터프로그램저작물 등록신청서

추가 알고리즘 명세 — 사용자 주도형 경주 시뮬레이션 수동 보정 모듈

기본 사항 (기존 등록 정보와 동일)

항목	내용
저작물명	경마통 Race Master — 실시간 AI 경마 레이스 시뮬레이터
추가 알고리즘명	사용자 주도형 경주 시뮬레이션 실시간 배팅 배당률 연동 수동 보정 엔진
개발 언어	Dart 3.9.2 / Flutter 3.35.4
운영환경	Android 15 (API 35), Web(Chrome/Firefox/Safari)
저작물 유형	컴퓨터프로그램저작물 (소스코드 + UI 레이아웃 명세)

제1조 알고리즘 개요 및 창작 배경

본 알고리즘은 기존 등록된 「경마통 Race Master」의 물리 엔진(GridRail 4존 엔진 및 Jockey Engine)에, **사용자가 UI를 통해 개별 경주마의 물리 파라미터를 실시간으로 수정·주입(inject)할 수 있는 인터페이스 레이어**를 추가한 독립적 컴퓨터프로그램저작물이다.

기존 경마 시뮬레이터는 KRA(한국마사회) 공공 API에서 수신한 데이터를 기반으로 자동 연산하는 구조였으나, 본 모듈은 **배당률 가중치, 기수 멘탈 상태, 스피트 시점**이라는 3가지 핵심 변수를 유저가 직접 보정할 수 있는 「수동 변수 보정 패널(Manual Calibration Panel)」과, 이 입력값을 물리 엔진의 P_final 다중 멀티플라이어 체인에 실시간으로 바인딩하는 「API 파라미터 오버라이딩 제어 계층(UserCalibrationController)」을 새롭게 창작하였다.

제2조 UI 레이아웃 구조 명세

2-1. 수동 변수 보정 패널 평면 배치

수동 변수 보정 패널은 기존 배당률 표(Odds Table)와 동일 화면에 나란히 배치되는 독립 컨테이너(Container)로 구성되며, 다음과 같은 계층적 위젯 트리 구조를 가진다.

```

ManualCalibrationPanel (StatelessWidget)
|
├ Container (배경: #0E1E3A, 테두리: Gold #33FFD700, radius: 16px)
|   └ Column (crossAxisAlignment: stretch)
|       |
|       └ [1] _PanelHeader (Row)
|           └ Icon(tune_rounded) + Text('수동 변수 보정 패널')
|           └ 활성 보정수 배치 (Container: Gold #FFD700, '${N}마 보정중')
|               └ TextButton('전체 초기화') ← ctrl.resetAll() 트리거
|
├ Divider (구분선)
|
├ [2] _HorseCalibTile × N마 (마번별 보정 행, Column)
|   └ _GateNoChip          - 마번 번호 칩 (색상 순환 10색)
|   └ CalibrationOddsField - ① 배당률 가중치 입력창
|   └ CalibrationJockeySlider - ② 기수 멘탈 보정 슬라이더
|   └ CalibrationSpurtDropdown - ③ 스퍼트 시점 드롭다운
|
└ [3] _PanelFooter (Row)
    └ Icon(check_circle / lock)
    └ Text('보정값 실시간 반영 중 · 마지막 업데이트 HH:MM:SS')

```

2-2. 개별 UI 컴포넌트 평면 배치

각 `_HorseCalibTile` 은 수직(Column) 레이아웃으로 아래 3개 컴포넌트를 순서대로 배치한다:

```

_HorseCalibTile (gateNo=N)
├─ Row [마번칩][말이름·배당·기수명][초기화아이콘]
│
├─ CalibrationOddsField (Row)
│   ├── Text('배당 가중치', width: 80)
│   ├── TextFormField (TextInputType.numberWithOptions)
│   │   ├── 입력 범위: -2.0 ~ +2.0 (소수점 1자리)
│   │   ├── inputFormatters: [FilteringTextInputFormatter]
│   │   ├── 실시간 색상: 양수=초록(#66BB6A), 음수=빨강(#EF5350)
│   │   └─ onChanged → ctrl.setOddsWeight(gateNo, value)
│   └─ Text('±2.0')
│
├─ CalibrationJockeySlider (Row)
│   ├── Text('기수 멘탈', width: 80)
│   ├── Text('패널티')
│   ├── Slider (min:-1.0, max:+1.0, divisions:20)
│   │   ├── SliderTheme (activeTrackColor: 동적 색상)
│   │   └─ onChanged → ctrl.setJockeyBuff(gateNo, value)
│   ├── Text('버프')
│   └─ Text('+N%' / '-N%' / '기본')
│
└─ CalibrationSpurtDropdown (Row)
    ├── Text('스퍼트 시점', width: 80)
    └─ DropdownButton<SpurtTimingSlot>
        ├── 초기 스퍼트 (-50m)
        ├── 표준 (기본값)
        ├── 후기 스퍼트 (+40m)
        └─ 극후기 (+80m)
        └─ onChanged → ctrl.setSpurtTiming(gateNo, slot)

```

2-3. 데이터 바인딩 트리 구조

UI 컴포넌트에서 물리 엔진까지의 완전한 데이터 바인딩 경로:

```

[사용자 입력]
CalibrationOddsField.onChange(value)
  → UserCalibrationController.setOddsWeight(gateNo, value)
  → RaceCalibrationSnapshot.update(HorseCalibration(userOddsWeight))
  → ChangeNotifier.notifyListeners()
  → Consumer<UserCalibrationController>.builder 리빌드 (16ms 이내)

[물리 엔진 반영 - 매 틱 16ms]
_initHorses():
  base × ctrl.oddsAdjFactor(gateNo) → calibratedBase (baseSpeed 보정)
  ctrl.spurtProgOffset(gateNo) → _horseSpurtOffset[gateNo]

_onTick() → inZone4Straight:
  ctrl.speedMultOverride(gateNo, afScale) → speedMult × (1.0 ± Δ)

applyToEntry(entry):
  newUserBonus = entry.userBonus
    + (userOddsWeight × 0.015 × 333) ← oddsAdj
    + (userJockeyBuff × 0.12 × 333) ← jockeyAdj
  → .clamp(-5.0, +5.0)
  → _Horse.userBonus → Zone3 speedMult *= 1.12 + bf×0.08×user

```

제3조 핵심 소스코드 명세 — user_calibration_controller.dart

3-1. 파일 구성 요약

파일명	위치	역할
user_calibration_model.dart	lib/models/	보정 데이터 DTO 계층
user_calibration_controller.dart	lib/providers/	물리 엔진 오버라이딩 제어
manual_calibration_panel.dart	lib/widgets/	UI 컴포넌트 집합

3-2. 핵심 데이터 모델 — SpurtTimingSlot 열거형

```

// 스퍼트 시점 드롭다운 선택값 → Zone4 진입 prog 오프셋으로 변환
enum SpurtTimingSlot {
  early ('조기 스퍼트', -0.05), // 기준보다 50m 이른 스퍼트 발동
  standard('표준', 0.0), // KRA 기본 스퍼트 시점 (변경 없음)
  delayed ('후기 스퍼트', 0.04), // 기준보다 40m 늦은 스퍼트 발동
  veryLate('극후기', 0.08); // 기준보다 80m 늦은 스퍼트 발동

  const SpurtTimingSlot(this.label, this.progOffset);
  final String label;
  final double progOffset; // prog 단위 오프셋 (0.01 ≈ 경주거리의 1%)
}

```

3-3. 핵심 데이터 모델 — HorseCalibration 불변 객체

```
// 마번 1개의 보정 데이터 집합 (불변 value object)
@immutable
class HorseCalibration {
  final int gateNo; // 마번 식별자 (1-based)
  final double userOddsWeight; // 배당률 가중치: -2.0 ~ +2.0 (기본 0.0)
  final double userJockeyBuff; // 기수 멘탈 보정: -1.0 ~ +1.0 (기본 0.0)
  final SpurtTimingSlot spurtTiming; // 스퍼트 시점 (기본 standard)

  bool get isDefault =>
    userOddsWeight == 0.0 &&
    userJockeyBuff == 0.0 &&
    spurtTiming == SpurtTimingSlot.standard;
}
```

3-4. 핵심 제어 메서드 — API 파라미터 오버라이딩 연산식

(1) setOddsWeight — 배당률 가중치 실시간 주입

```
void setOddsWeight(int gateNo, double value) {
  // ① 범위 검증 및 클램프 (-2.0 ~ +2.0)
  final clamped = value.clamp(kOddsWeightMin, kOddsWeightMax);
  // ② 불변 스냅샷 갱신 (Dart immutable update pattern)
  _snapshot = _snapshot.update(current.copyWith(userOddsWeight: clamped));
  // ③ ChangeNotifier 전파 → UI 즉시 갱신 + 물리 루프 동기화
  notifyListeners();
}
```

물리 엔진 반영 경로 (oddsAdjFactor):

```
Δfactor = userOddsWeight × kOddsWeightSpeedDelta(0.015)
baseSpeed_final = baseSpeed_KRA × (1.0 + Δfactor)
유효 범위: baseSpeed × [0.970, 1.030]
```

(2) setJockeyBuff — 기수 멘탈 슬라이더 실시간 주입

```
void setJockeyBuff(int gateNo, double value) {
  final clamped = value.clamp(kJockeyBuffMin, kJockeyBuffMax);
  _snapshot = _snapshot.update(current.copyWith(userJockeyBuff: clamped));
  notifyListeners();
}
```

물리 엔진 반영 경로 (speedMultOverride):

```
Δmult = userJockeyBuff × kJockeyBuffMaxDelta(0.12) × afScale(1.0~1.35)
speedMult_Zone4 *= (1.0 + Δmult)
유효 범위: speedMult × [0.838, 1.162] (afScale=1.0 기준)
유효 범위: speedMult × [0.784, 1.216] (afScale=1.35, 9경주 기준)
```

(3) setSpurtTiming — 스퍼트 시점 드롭다운 주입

```
void setSpurtTiming(int gateNo, SpurtTimingSlot slot) {  
    _snapshot = _snapshot.update(current.copyWith(spurtTiming: slot));  
    notifyListeners();  
}
```

물리 엔진 반영 경로 (spurtProgOffset):

```
_horseSpurtOffset[gateNo] = slot.progOffset (-0.05 ~ +0.08)
```

적용 시점: `_initHorses()` — 경주 시작 전 1회 계산

효과:

early(-0.05): 기본 스퍼트 시점보다 5% 일찍 가속 → 선행말 유리

standard(0.0): 변경 없음

delayed(+0.04): 기본보다 4% 늦게 가속 → 추입형 후방 배치마 유리

veryLate(+0.08): 기본보다 8% 늦게 가속 → 극단적 후방 대기 전략

(4) applyToEntry — P_final 전체 오버라이딩 합산식

```
HorseEntry applyToEntry(HorseEntry entry) {  
    if (calib.isDefault) return entry; // 보정 없으면 zero-cost 반환  
  
    // ① 배당률 가중치 → userBonus 환산  
    // userOddsWeight +2.0 → userBonus +10.0 (스케일: ×333)  
    final oddsAdj = calib.userOddsWeight × kOddsWeightSpeedDelta × 333.0;  
  
    // ② 기수 멘탈 슬라이더 → userBonus 환산  
    // userJockeyBuff +1.0 → userBonus +40.0 (스케일: ×333)  
    final jockeyAdj = calib.userJockeyBuff × kJockeyBuffMaxDelta × 333.0;  
  
    // ③ 합산 후 물리 엔진 userBonus 범위(-5.0~+5.0) 클램프  
    final newUserBonus = (entry.userBonus + oddsAdj + jockeyAdj).clamp(-5.0, 5.0);  
  
    return entry.copyWith(userBonus: newUserBonus);  
}
```

P_final 전체 연산 흐름 (사용자 보정 포함 완전판):

```
P_final = baseSpeed × oddsAdjFactor(userOddsWeight)  
          × blockMult × cornerPenalty × boostMult × spurtMult  
          × g1fBoostMult × safeModeCoeff × mentalBuffCoeff  
          × surgeBuffCoeff × speedMultOverride(userJockeyBuff, afScale)  
          + noise(±0.00002)
```

여기서:

oddsAdjFactor = $1.0 + \text{userOddsWeight} \times 0.015$ ∈ [0.970, 1.030]

speedMultOverride = $1.0 + \text{userJockeyBuff} \times 0.12 \times \text{afScale}$ ∈ [0.784, 1.216]

제4조 기능 명세서 — 실시간 배팅 배당률 연동 및 사용자 주도형 수동 보정

4-1. 기능 정의

기능명: 실시간 배팅 배당률 연동 사용자 주도형 경주 시뮬레이션 데이터 수동 보정 기능

기능 구분: KRA API 파라미터 오버라이딩(Parameter Overriding) 기반 실시간 물리 엔진 재조정 인터페이스

작동 원리:

1. KRA 공공 API로부터 수신한 배당률(odds), 기수 통산승률(jockeyRcWins), 마별 G1F 성적(g1fRating) 등의 원시 데이터를 기반으로 물리 엔진이 초기 파라미터를 산출한다.
2. 사용자가 「수동 변수 보정 패널」에서 다음 3종 변수를 직접 수정한다:
 - **userOddsWeight (배당률 가중치):** -2.0~+2.0 범위의 부동소수점 값. 배당률 기반 기초 속도 산출 계수를 $\pm 3\%$ 범위에서 오버라이드한다.
 - **userJockeyBuff (기수 멘탈 보정):** -1.0~+1.0 범위의 슬라이더 값. Zone4(후반 400m~결승선) 가속도 계수를 $\pm 12\% \times \text{afScale}$ 범위에서 오버라이드한다.
 - **spurtTiming (스퍼트 시점):** 4단계 드롭다운 선택값. Zone4 스퍼트 발동 prog 오프셋을 -5%~+8% 범위에서 개별 조정한다.
3. UserCalibrationController.notifyListeners()가 즉시 호출되어 물리 루프(16ms 주기)의 다음 틱에서 보정값이 반영된다.
4. 경주 시작 시점에 lockPanel()이 호출되어 보정 패널이 잠기며, 경주 종료 후 unlockPanel()로 해제된다.

4-2. API 파라미터 오버라이딩 기술 명세

오버라이딩 대상 API 파라미터 목록:

파라미터명	API 출처	기본 연산식	오버라이드 방식	유효 범위
baseSpeed	KRA rcWins, jockeyRcWins	$\text{rcWins} \times 50 + \text{jockeyRcWins} \times 30 + (60 - \text{wgBudam}) \times 0.2$	$\times \text{oddsAdjFactor}$	$\pm 3\%$
speedMult_Zone4	내부 계산	$\text{g1fBoostMult} \times \text{safeModeCoeff} \times \dots$	$\times \text{speedMultOverride}$	$\pm 12 \sim 16\%$
_spurt100	거리 기반 계산	$\text{goalP} - 100\text{m} / \text{총거리}$	$+ \text{spurtProgOffset}$	-5%~+8%
userBonus	KRA userBonusScore	원시 KRA 값	$+ (\text{oddsAdj} + \text{jockeyAdj})$	± 5.0

오버라이딩 적용 시점:

파라미터	적용 시점	적용 메서드
baseSpeed 보정	경주 시작 시 1회 (_initHorses)	oddsAdjFactor(gateNo)
spurt 오프셋	경주 시작 시 1회 (_initHorses)	spurtProgOffset(gateNo)
Zone4 speedMult	매 틱 16ms (_onTick)	speedMultOverride(gateNo, afScale)
userBonus 합산	매 틱 16ms (_onTick)	applyToEntry(entry)

4-3. ChangeNotifier 기반 리액티브 상태 관리 명세

```

UserCalibrationController (ChangeNotifier)
|
|─ RaceCalibrationSnapshot ← 불변(immutable) 보정 스냅샷
|   Map<int, HorseCalibration> → gateNo당 HorseCalibration 1개
|
|─ setOddsWeight() → notifyListeners() → Consumer 리빌드 (≤16ms)
|─ setJockeyBuff() → notifyListeners() → Consumer 리빌드 (≤16ms)
|─ setSpurtTiming() → notifyListeners() → Consumer 리빌드 (≤16ms)
|
|─ lockPanel() ← 경주 시작 시 호출 → panelEnabled = false
|─ unlockPanel() ← 경주 종료 시 호출 → panelEnabled = true

```

스레드 안전성: Flutter의 단일 스레드(Dart isolate) 모델에 의해 경쟁 조건(race condition) 없이 안전하게 동작한다. notifyListeners()는 위젯 rebuild 큐에 등록되며, 물리 루프(Timer.periodic)와 동일 isolate에서 순차 실행된다.

4-4. 창작성 진술

본 알고리즘의 독창적 창작 요소는 다음과 같다:

- 3변수 오버라이딩 구조의 독창성:** 배당률 가중치, 기수 멘탈, 스퍼트 시점이라는 서로 다른 물리적 의미를 가진 3가지 변수를 단일 HorseCalibration 불변 객체로 캡슐화하고, 이를 P_final 8중 멀티플라이어 체인의 서로 다른 위치에 선택적으로 주입하는 구조는 타 경마 시뮬레이터에 존재하지 않는 독자적 설계이다.
- 비선형 스케일 팩터(×333)의 창작성:** userOddsWeight(-2.0~+2.0)와 userJockeyBuff(-1.0~+1.0)를 userBonus(-5.0~+5.0) 스케일로 역변환하기 위해 333을 스케일 팩터로 채택한 것은, 기존 물리 엔진의 userBonus 입력 범위(-5~+5)와의 호환성을 유지하면서도 UI 입력 범위(±2.0, ±1.0)를 직관적으로 설계한 독창적 수치 설계이다.
- 패널 잠금/해제 메커니즘의 창작성:** 경주 시작(lockPanel) → 경주 종료(unlockPanel)의 생명주기 연동은, 경주 도중 파라미터 조작으로 인한 물리 엔진 불안정을 방지하는 동시에 결과 확인 후 즉시 다음 경주 보정을 준비할 수 있도록 설계된 창작적 상태 기계(state machine)이다.
- SpurtTimingSlot 열거형의 도박론적 설계:** 조기(-50m)/표준/후기(+40m)/극후기(+80m) 4단계 스퍼트 시점 분류는, 실제 경마에서 관찰되는 선행마·중간마·후방마의 전략적 행동 패턴을 prog 오프셋으로 추상화한 것으로, 현실 경마 데이터 분석에 기반한 독창적 모델링이다.

5. **Zero-cost 최적화 패턴의 창작성:** `if (calib.isDefault) return entry;` 구문으로, 보정값이 없는 경우 새 객체 생성을 완전히 생략하는 zero-cost 반환 패턴은, 16ms 물리 루프에서 발생하는 GC 압력을 최소화하기 위한 Flutter 특화 최적화 기법이다.
6. **불변 스냅샷 갱신 패턴(Immutable Update Pattern)의 적용:** Dart의 불변 객체 + `copyWith` 패턴을 활용하여, 보정 상태의 이전 스냅샷을 보존하면서 특정 마번만 선택적으로 갱신하는 `RaceCalibrationSnapshot.update()` 구조는 Flutter `ChangeNotifier`와 결합된 독창적 상태 관리 설계이다.

제5조 소스코드 추출본 전문

5-1. `user_calibration_model.dart` (핵심 발췌)

```
// — SpurtTimingSlot: 스퍼트 시점 열거형 —
enum SpurtTimingSlot {
  early   ('조기 스퍼트', -0.05),
  standard('표준',      0.0),
  delayed ('후기 스퍼트', 0.04),
  veryLate('극후기',    0.08);

  const SpurtTimingSlot(this.label, this.progOffset);
  final String label;
  final double progOffset;
}

// — HorseCalibration: 마번 1개의 보정 데이터 집합 —
@immutable
class HorseCalibration {
  final int gateNo;
  final double userOddsWeight; // -2.0 ~ +2.0
  final double userJockeyBuff; // -1.0 ~ +1.0
  final SpurtTimingSlot spurtTiming;

  const HorseCalibration({
    required this.gateNo,
    this.userOddsWeight = 0.0,
    this.userJockeyBuff = 0.0,
    this.spurtTiming     = SpurtTimingSlot.standard,
  });
  bool get isDefault =>
    userOddsWeight == 0.0 && userJockeyBuff == 0.0 &&
    spurtTiming == SpurtTimingSlot.standard;
}
```

5-2. user_calibration_controller.dart (핵심 발췌)

```
// — 물리 엔진 오버라이드 상수 —
const double kOddsWeightSpeedDelta = 0.015; // ±3% baseSpeed 보정
const double kJockeyBuffMaxDelta = 0.12; // ±12%×afScale speedMult 보정

// — API 파라미터 오버라이딩 핵심 메서드 —

/// [1] baseSpeed oddsAdjFactor: 배당률 가중치 → 기초속도 보정 계수
double oddsAdjFactor(int gateNo) {
  final calib = _snapshot.forGate(gateNo);
  if (calib.userOddsWeight == 0.0) return 1.0; // zero-cost
  return (1.0 + calib.userOddsWeight * kOddsWeightSpeedDelta).clamp(0.97, 1.03);
}

/// [2] Zone4 speedMultOverride: 기수 멘탈 슬라이더 → 가속도 계수 오버라이드
double speedMultOverride(int gateNo, double afScale) {
  final calib = _snapshot.forGate(gateNo);
  if (calib.userJockeyBuff == 0.0) return 1.0; // zero-cost
  final delta = calib.userJockeyBuff * kJockeyBuffMaxDelta * afScale;
  return (1.0 + delta).clamp(0.7, 1.35);
}

/// [3] spurtProgOffset: 스퍼트 시점 드롭다운 → Zone4 진입 오프셋
double spurtProgOffset(int gateNo) =>
  _snapshot.forGate(gateNo).spurtTiming.progOffset;

/// [4] applyToEntry: P_final 전체 오버라이딩 합산
HorseEntry applyToEntry(HorseEntry entry) {
  final calib = _snapshot.forGate(entry.gateNo);
  if (calib.isDefault) return entry; // zero-cost 최적화
  final oddsAdj = calib.userOddsWeight * kOddsWeightSpeedDelta * 333.0;
  final jockeyAdj = calib.userJockeyBuff * kJockeyBuffMaxDelta * 333.0;
  final newUserBonus = (entry.userBonus + oddsAdj + jockeyAdj).clamp(-5.0, 5.0);
  return entry.copyWith(userBonus: newUserBonus);
}
```

5-3. manual_calibration_panel.dart (UI 컴포넌트 핵심 발췌)

```
// — CalibrationOddsField: 배당률 가중치 입력창 —
TextFormField(
  keyboardType: TextInputType.numberWithOptions(decimal: true, signed: true),
  inputFormatters: [FilteringTextInputFormatter.allow(RegExp(r'^-?\d{0,1}\.?\d{0,1}$'))],
  onChanged: (v) {
    final parsed = double.tryParse(v);
    if (parsed != null) ctrl.setOddsWeight(widget.gateNo, parsed);
  },
)

// — CalibrationJockeySlider: 기수 멘탈 보정 슬라이더 —
Slider(
  value: val.clamp(kJockeyBuffMin, kJockeyBuffMax),
  min: -1.0, max: 1.0, divisions: 20,
  onChanged: enabled ? (v) => ctrl.setJockeyBuff(gateNo, v) : null,
)

// — CalibrationSpurtDropdown: 스퍼트 시점 드롭다운 —
DropdownButton<SpurtTimingSlot>(
  value: current,
  onChanged: enabled ? (slot) {
    if (slot != null) ctrl.setSpurtTiming(gateNo, slot);
  } : null,
  items: SpurtTimingSlot.values.map((slot) =>
    DropdownMenuItem(value: slot, child: Text(slot.label))
  ).toList(),
)
```

제6조 물리 엔진 연동 위치 명세 (race_animation_screen.dart)

6-1. _initHorses() — 경주 시작 시 1회 적용

```
void _initHorses() {
  // [UserCalib-1] oddsAdjFactor: baseSpeed 배당률 가중치 보정
  final calibratedBase = base * _calibCtrl.oddsAdjFactor(h.gateNo);
  //   userOddsWeight +2.0 → baseSpeed × 1.030 (최대 +3%)
  //   userOddsWeight -2.0 → baseSpeed × 0.970 (최대 -3%)

  // [UserCalib-2] spurtProgOffset: 스퍼트 시점 개별 오프셋 캐싱
  _horseSpurtOffset[h.gateNo] = _calibCtrl.spurtProgOffset(h.gateNo);
}
```

6-2. _onTick() → inZone4Straight — 매 틱 16ms 적용

```
if (inZone4Straight) {  
  // ... (기존 Jockey Engine 3중 적용) ...  
  
  // [UserCalib-3] speedMultOverride: 기수 멘탈 → Zone4 가속도 오버라이드  
  final userMultOverride = _calibCtrl.speedMultOverride(h.entry.gateNo, afScale);  
  if (userMultOverride != 1.0) {  
    speedMult *= userMultOverride;  
    // 버프 시 글로우 이펙트 추가  
    if (userMultOverride > 1.0 && !h.boostActive) {  
      h.boostGlow = (h.boostGlow + (userMultOverride - 1.0) * 0.5).clamp(0, 1);  
    }  
  }  
}
```

제7조 버전 이력

버전	날짜	변경 내용
v1.0.0	2025-05-22	초기 구현: UserCalibrationController, HorseCalibration, ManualCalibrationPanel
v1.0.0	2025-05-22	race_animation_screen.dart 연동: oddsAdjFactor, speedMultOverride, spurtProgOffset
v1.0.0	2025-05-22	flutter analyze 0 errors 확인 완료

본 문서는 한국저작권위원회 컴퓨터프로그램저작물 등록 신청을 위해 작성된 기술 명세서입니다.

소스코드의 무단 복제, 배포, 역공학 분석을 금합니다.

Copyright © 2025 경마통 Race Master. All rights reserved.